Bullets, aliens and hit detection, explained simply

Space Invaders on the DE2-115: how the three most important game rules work in the Verilog code

0. The big picture

The game has three kinds of moving things: your **bullet**, the **aliens**, and the **alien bullets**. None of them is a "picture that moves". Each is just a few numbers stored in registers (its x and y position, and an on/off flag). The screen is redrawn from those numbers for every pixel, so "moving" simply means "changing the numbers".

- **The tick.** All movement happens once every 10 ms, when the pulse **move_tick** arrives. Between ticks nothing changes.
- **Coordinates.** x counts from the left edge, y counts *downwards* from the top edge (so a bigger y is lower on the screen). The screen is 640 x 480 pixels.
- **Boxes.** For the rules, every object is treated as a rectangle (a "box"): alien 32 x 18 px, ship 40 x 16 px, bullets 4 x 14 px. Two things "touch" when their boxes overlap.
- **Alive.** An alien can be shot or can fire only if it is *alive*: it exists on screen (**present**) and has not been shot (**defeated** = 0).

Part	Question it answers	Main signals
1. Player bullet	How does my shot start, fly and end?	shoot, nes_a, nes_b, bullet_active, bullet_x, bullet_y
2. Alien logic	How do the aliens move and shoot?	apx_flat, apy_flat, x_offset, descent_offset, gen_alien[g], alien_bullet_*
3. Hit logic	What happens when things touch?	defeated, hit, score_reg, life_lost, lives_reg, touch, state

1. The player bullet

1.1 What is stored

Three registers describe your bullet: **bullet_active** (1 = a bullet is in the air), **bullet_x** (its left edge) and **bullet_y** (its *bottom* edge). The bullet is drawn as a box 4 px wide that extends 14 px upward from bullet_y. Only one player bullet can exist at a time.

1.2 Launching: detecting a new key press

The shoot key is held down for many clock cycles, but we want *one* bullet per press. The trick is to remember what the key was one clock ago. **nes_a** is the key now and **nes_b** is the key one clock earlier. A new press is "now = 1 and before = 0", which happens for exactly one clock when the key goes down.

launch (src/graphics.v, lines 665-666, 674-677)

```
nes_b <= nes_a;
nes_a <= shoot;

if (nes_a && !nes_b && !bullet_active) begin
    bullet_active <= 1'b1;
    bullet_x <= ship_x + (SHIP_W / 2) - (BULLET_W / 2);
    bullet_y <= SHIP_Y - 1;
```

When that happens (and no bullet is already flying), the bullet is switched on at the middle of the ship's top edge: x = ship_x + 20 - 2 so that the 4 px bullet is centred on the 40 px ship, and y = SHIP_Y - 1, just above the ship.

1.3 Flying and ending

Every move_tick, while the bullet is in the air, the code first runs the hit test (Part 3). If the bullet hit an alien it is switched off. If it did not hit anything but has climbed up to the top of the alien area (**top_y**) it is also switched off, which counts as a miss. Otherwise it moves up by **BULLET_SPEED** = 4 px. At 4 px per 10 ms the bullet travels 400 px per second, so it crosses the playfield in about one second.

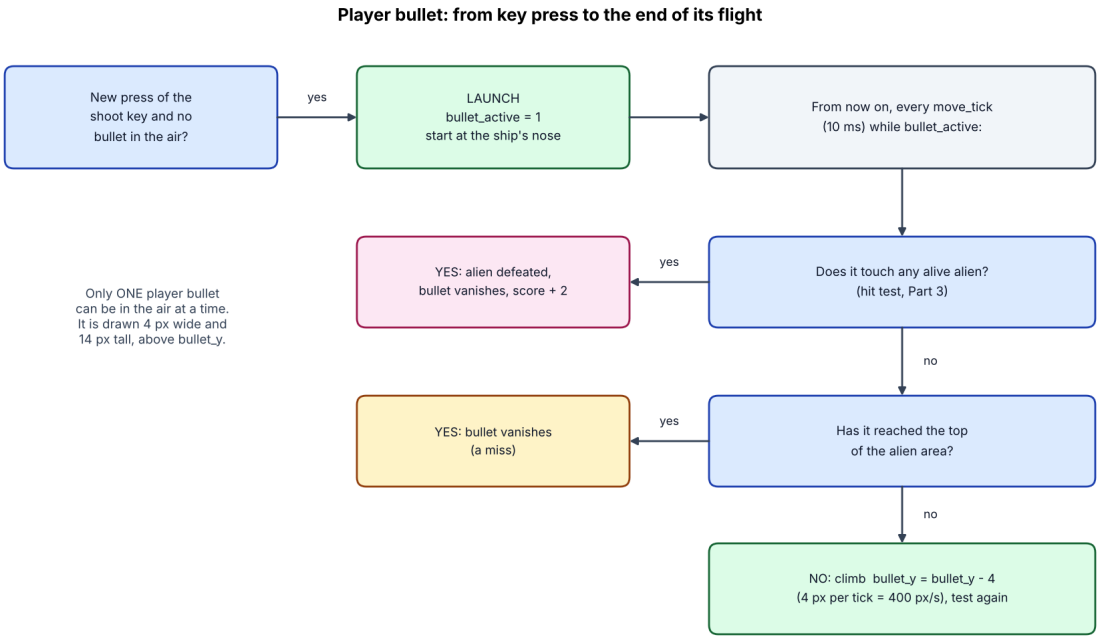


Figure 1. Life of the player bullet. All the decisions are taken once per move_tick.

flight (src/graphics.v, lines 678-698)

```
end else if (move_tick && bullet_active) begin
    for (k = 0; k < TOTAL_ALIENS; k = k + 1) begin
        cx = apx_flat[10*k +: 10];
        cy = apy_flat[10*k +: 10];
        if (!hit && alive[k] &&
            (bullet_x + BULLET_W >= cx) &&
            (bullet_x <= cx + ALIEN_W) &&
            (bullet_y >= cy) &&
            (bullet_y <= cy + ALIEN_H + BULLET_H)) begin
            defeated[k] <= 1'b1;
            hit = 1'b1;
        end
    end

    if (hit)
        bullet_active <= 1'b0;                          // consumed by a hit
    else if (bullet_y <= top_y)
        bullet_active <= 1'b0;                          // flew past every alien, missed
    else
        bullet_y <= bullet_y - BULLET_SPEED[9:0];       // keep travelling up
end
```

1.4 Drawing the bullet

The picture of the bullet is not stored anywhere. For each pixel the colour logic simply asks: “is this pixel inside the bullet's box?” If yes, it paints white.

src/graphics.v (src/graphics.v, lines 1239-1241)

```
bullet_hit = bullet_active &&
             (px_x >= bullet_x) && (px_x < bullet_x + BULLET_W) &&
             (px_y >= bullet_y - BULLET_H) && (px_y <= bullet_y);
```

2. The aliens

2.1 What an alien is in hardware

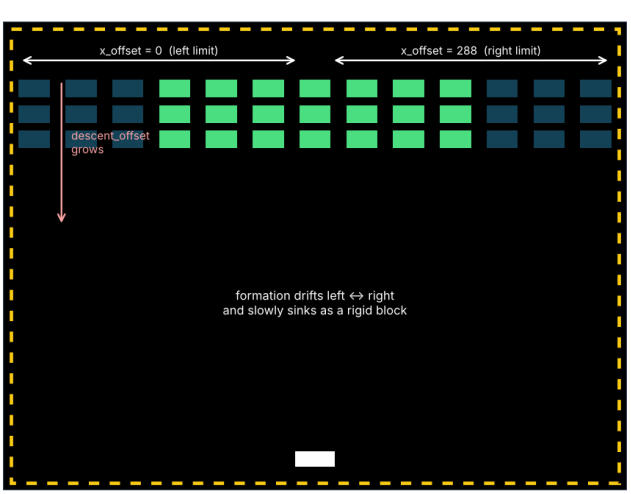
There are 21 alien *slots* (3 rows x 7 columns). An alien is not a separate circuit with its own picture; it is a slot number k with:

- a position: x in **apx_flat**, y in **apy_flat** (each slot has 10 bits of each, packed into one long vector);
- two flags: **present** (it exists on screen) and **defeated** (it was shot); **alive = present AND NOT defeated**;
- a colour bit **acol[k]** (green or magenta, random per game);
- the same shared 16 x 9 bitmap, drawn at double size (32 x 18 px) wherever its position says.

What differs between the three difficulty modes is only *how the position numbers change*.

2.2 EASY and MEDIUM: one flock

The 21 aliens move as a single rigid block. Only a few registers describe the whole block: **x_offset** (how far it has slid sideways) and **descent_offset** (how far it has sunk). Every alien's position is calculated from those and its own row and column.



Rigid formation (EASY / MEDIUM)

```
Alien (row R, col C) position:

  x = ALIEN_X0 + C*ALIEN_SPACING
        + x_offset - MAX_SWEEP
  y = ALIEN_Y0 + R*ROW_SPACING
        + descent_offset

ALIEN_X0 = 160   ALIEN_SPACING = 48
ALIEN_Y0 = 60    ROW_SPACING   = 26
MAX_SWEEP = 144

x_offset runs 0 .. 2*MAX_SWEEP = 288
(starts at 144 = centred), so the
leftmost alien can reach x = 16 and the
rightmost alien's right edge x = 624.

Only 2 shared counters + 3 offsets move
all 21 aliens: that is why it is 'rigid'.
```

Figure 2. The rigid formation and its position formula.

The movement is random but simple. Think of a **countdown timer**: when it reaches 0 the block takes one step and the timer is reloaded with a random number of ticks. Two timers run at the same time, one for going down and one for going sideways. At the left/right limit the direction flips, and it also flips at random about once in 256 steps, so the block wanders.

descent timer and sideways timer (src/graphics.v, lines 711-718, 727-748)

```
if (move_tick && !indep) begin
    if (descend_counter == 16'd0) begin
        descend_counter <= desc_delay(med, lfsr);
        descent_offset  <= descent_offset + desc_step(lfsr[7:6]);
    end else begin
        descend_counter <= descend_counter - 16'd1;
    end
end

if (move_tick && !indep) begin
    if (horiz_counter == 16'd0) begin
        horiz_counter <= horiz_delay(med, lfsr);
        if (move_dir_right) begin
            if (x_offset >= (2 * MAX_SWEEP))
                move_dir_right <= 1'b0;                 // hit the right limit: bounce
            else if ((lfsr[15:13] == 3'd0) && (lfsr[20:16] == 5'd0))
                move_dir_right <= 1'b0;                 // random reversal (1 in 256)
            else
                x_offset <= x_offset + 9'd1;
        end else begin
            if (x_offset == 9'd0)
                move_dir_right <= 1'b1;                 // hit the left limit: bounce
            else if ((lfsr[15:13] == 3'd0) && (lfsr[20:16] == 5'd0))
                move_dir_right <= 1'b1;                 // random reversal (1 in 256)
            else
                x_offset <= x_offset - 9'd1;
        end
    end else begin
        horiz_counter <= horiz_counter - 16'd1;
    end
end
```

2.3 HARD: 21 individuals

In HARD mode there is no block. Each alien has its own little machine with its own random generator, its own position and its own timers. It has two states: **WAIT** (not on screen yet; counts down to its appearance time) and **ROAM** (on screen; takes small random steps sideways and down on its own timers). No alien reads another alien's numbers, so they cannot move together. They always stay inside the play rectangle, and the rectangle's bottom edge is the ship's row.

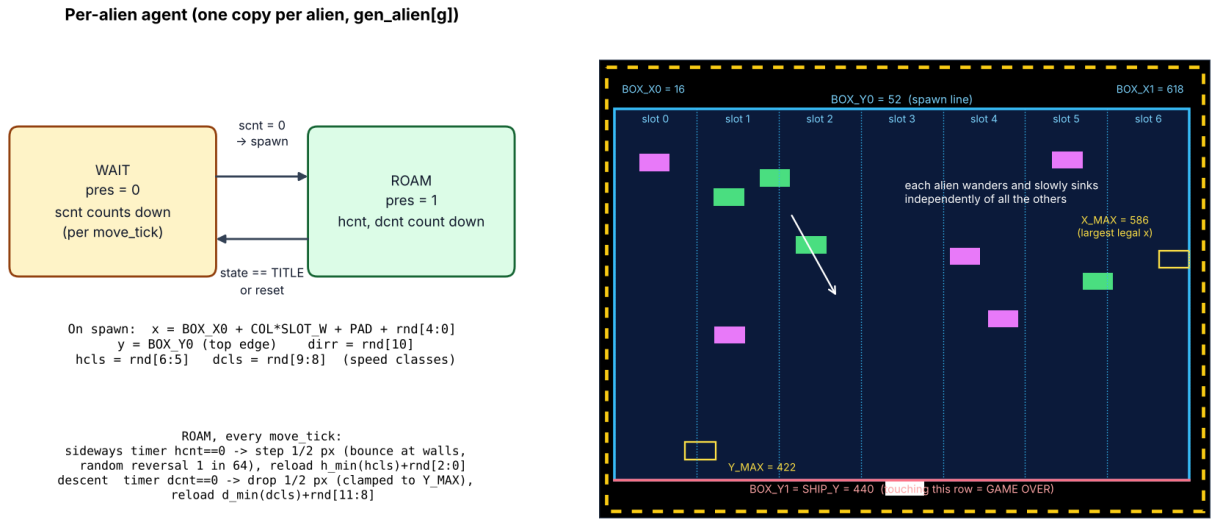


Figure 3. The WAIT / ROAM machine of one alien and the play rectangle.

ROAM state of one alien (src/graphics.v, lines 517-540)

```
end else if (!defeated[g]) begin
    //---------------- ROAM: sideways
    if (hcnt == 10'd0) begin
        hcnt <= h_min(hcls) + rnd[2:0];
        if (dirr) begin
            if (x_fwd > X_MAX)        dirr <= 1'b0;      // right wall: bounce
            else if ((rnd[6:4] == 3'd0) && (rnd[9:7] == 3'd0)) dirr <= 1'b0;   // random reversal (1 in 64)
            else                      x <= x_fwd;
        end else begin
            if (x < BOX_X0 + hstep)    dirr <= 1'b1;      // left wall: bounce
            else if ((rnd[6:4] == 3'd0) && (rnd[9:7] == 3'd0)) dirr <= 1'b1;   // random reversal (1 in 64)
            else                      x <= x - hstep;
        end
    end else begin
        hcnt <= hcnt - 10'd1;
    end
    //---------------- ROAM: descent
    if (dcnt == 10'd0) begin
        dcnt <= d_min(dcls) + rnd[11:8];
        y    <= (y_fwd > Y_MAX) ? Y_MAX : y_fwd;
    end else begin
        dcnt <= dcnt - 10'd1;
    end
end
```

2.4 How the aliens shoot

Two shooters (A and B) work independently, each with its own random countdown and its own bullet. When the countdown reaches 0: a random alien index from 0 to 20 is picked; if that alien is alive and the shooter's bullet is not already flying, a bullet starts from the bottom centre of that alien; if the alien is not alive, another one is tried on the very next tick.

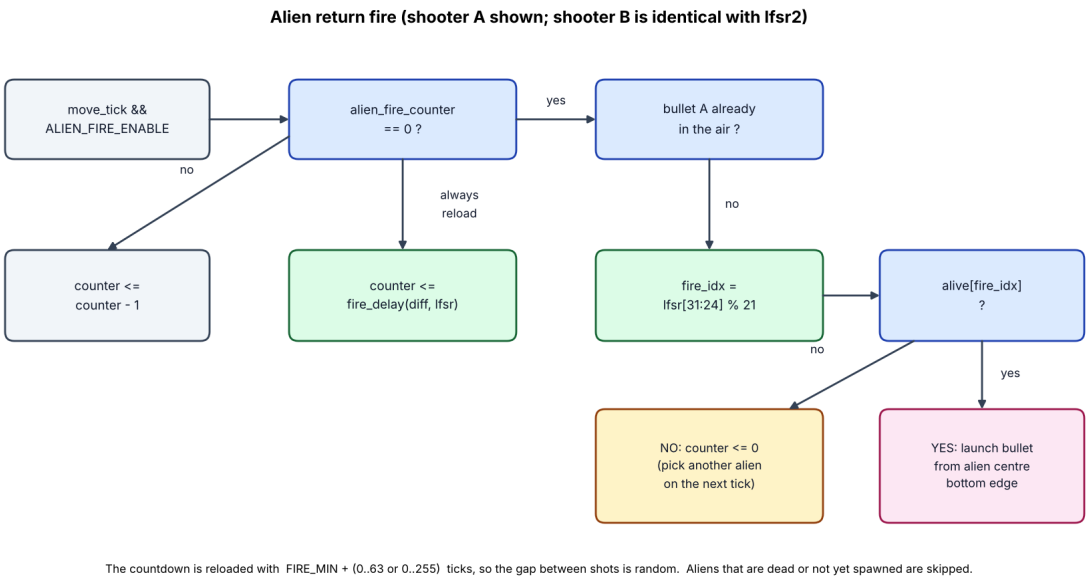


Figure 4. Alien fire logic (shooter A; shooter B is identical).

shooter A (src/graphics.v, lines 760-776)

```
if (move_tick && (ALIEN_FIRE_ENABLE != 0)) begin
    if (alien_fire_counter == 16'd0) begin
        alien_fire_counter <= fire_delay(diff, lfsr);
        if (!alien_bullet_active) begin
            fire_idx = lfsr[31:24] % TOTAL_ALIENS;
            if (!alive[fire_idx]) begin
                alien_fire_counter <= 16'd0;   // dead / absent pick: try again next tick
            end else begin
                alien_bullet_active <= 1'b1;
                alien_bullet_x <= apx_flat[10*fire_idx +: 10] + (ALIEN_W / 2) - (BULLET_W / 2);
                alien_bullet_y <= apy_flat[10*fire_idx +: 10] + ALIEN_H;
            end
        end
    end else begin
        alien_fire_counter <= alien_fire_counter - 16'd1;
    end
end
```

	EASY	MEDIUM	HARD
Movement	one rigid block	one rigid block	each alien on its own
Average descent	about 4 px/s	about 12 px/s	about 10 / 8 / 6 px/s (3 classes)
Average sideways speed	about 2.5 px/s	about 10.5 px/s	about 20 / 14 / 10 px/s
Time between shots (each shooter)	about 3 s	about 0.9 s	about 0.8 s
Alien bullet speed	400 px/s	600 px/s	600 px/s

3. Hit logic: what happens when things touch

There are four situations. All of them are just “do two boxes overlap?” checks, done inside the same clocked block as the movement.

3.1 Your bullet hits an alien

Two boxes overlap when they overlap *both* horizontally and vertically. For the bullet (4 x 14, y = bottom edge) and an alien at (cx, cy) of size 32 x 18, that gives four conditions, shown below. The vertical window is 18 (alien) + 14 (bullet) = 32 px tall, and the bullet only moves 4 px per tick, so it can never skip over an alien between two checks.

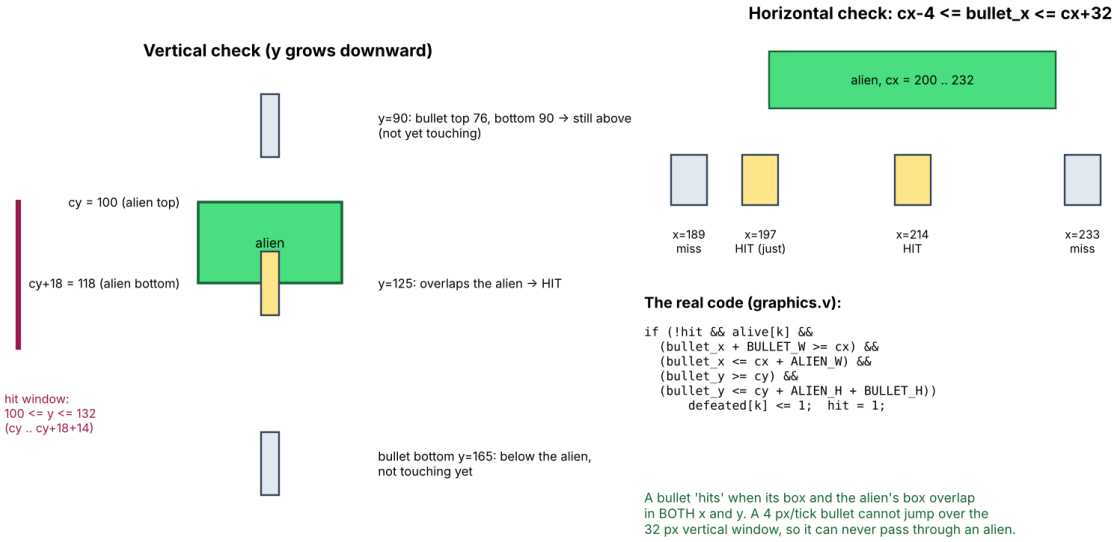


Figure 5. The bullet-versus-alien overlap test, with a numeric example.

Worked example. Alien at cx = 200, cy = 100. Bullet at bullet_x = 210, bullet_y = 120. Check: 210 + 4 = 214 >= 200 (true), 210 <= 232 (true), 120 >= 100 (true), 120 <= 132 (true). All four hold, so it is a hit. If the bullet were at y = 140 it would be too low (140 > 132) and would miss, and at y = 90 it would not have reached the alien yet.

the hit test (checked for all 21 aliens) (src/graphics.v, lines 679-691)

```
for (k = 0; k < TOTAL_ALIENS; k = k + 1) begin
    cx = apx_flat[10*k +: 10];
    cy = apy_flat[10*k +: 10];
    if (!hit && alive[k] &&
        (bullet_x + BULLET_W >= cx) &&
        (bullet_x <= cx + ALIEN_W) &&
        (bullet_y >= cy) &&
        (bullet_y <= cy + ALIEN_H + BULLET_H)) begin
        defeated[k] <= 1'b1;
        hit = 1'b1;
    end
end
```

What a hit does: the alien's **defeated** bit is set (it stops being drawn, can no longer be hit or fire), **hit** becomes 1 which switches the bullet off and adds 2 to the score. The **!hit** in the condition makes the loop stop at the first alien found, so if two overlapping aliens are under the bullet in HARD mode only the lower-numbered one is destroyed.

3.2 An alien bullet hits your ship

The same overlap test, but with the alien bullet and the ship's box (x from ship_x to ship_x + 40, y from 440 to 456). A bullet that reaches the bottom frame without touching the ship (y + 14 >= FLOOR_Y = 468) just disappears.

alien bullet A flight and ship hit (B is identical) (src/graphics.v, lines 805-815)

```
if (alien_bullet_active && move_tick) begin
    if ((alien_bullet_x + BULLET_W >= ship_x) && (alien_bullet_x <= ship_x + SHIP_W) &&
        (alien_bullet_y + BULLET_H >= SHIP_Y) && (alien_bullet_y <= SHIP_Y + SHIP_H)) begin
        alien_bullet_active <= 1'b0;
        life_lost = 1'b1;
    end else if (alien_bullet_y + BULLET_H >= FLOOR_Y) begin
        alien_bullet_active <= 1'b0;   // missed: gone just before the bottom frame
    end else begin
        alien_bullet_y <= alien_bullet_y + abullet_speed;
    end
end
```

A hit costs one life through **life_lost**. The lives go 3, 2, 1 and then 0. If lives would reach 0 the game ends. Otherwise the ship goes back to the centre of the screen and all bullets are cleared, but the aliens stay exactly where they are.

lives (src/graphics.v, lines 849-860)

```
if (life_lost) begin
    if (lives_reg <= 2'd1) begin
        lives_reg <= 2'd0;
        state     <= S_OVER;
    end else begin
```

```
        lives_reg           <= lives_reg - 2'd1;
        life_lost_pending   <= 1'b1;
        bullet_active       <= 1'b0;
        alien_bullet_active <= 1'b0;
        alien_bullet_b_active <= 1'b0;
    end
end
```

Lives before the hit	Lives after	Result
3	2	ship recentred, bullets cleared, play continues
2	1	same
1	0	state = S_OVER, GAME OVER screen

3.3 An alien reaches your row

If *any* alive alien's bottom edge reaches the ship's row (y + 18 >= 440), the game is over immediately, whatever number of lives is left. This check is written last in the block so it overrides anything else that happens in the same clock.

src/graphics.v (src/graphics.v, lines 866-875)

```
touch = 1'b0;
for (k = 0; k < TOTAL_ALIENS; k = k + 1) begin
    cy = apy_flat[10*k +: 10];
    if (alive[k] && (cy + ALIEN_H >= SHIP_Y))
        touch = 1'b1;
end
if (touch) begin
    lives_reg <= 2'd0;
    state     <= S_OVER;
end
```

3.4 You shoot the last alien

defeated has one bit per alien. The expression **&defeated** (AND of all 21 bits) is 1 only when every alien has been shot, and then the game goes to S_WON (GAME WON).

src/graphics.v (src/graphics.v, lines 839-842)

```
if (!flip_counter && &defeated) begin
    level_reg <= level_reg + 2'd1;
    state     <= S_WON;
end
```

3.5 Summary of all events

Event	How it is detected	Result
Player bullet touches an alive alien	box overlap in x and y, alien alive	alien defeated, bullet off, score +2
Player bullet reaches the top	bullet_y <= top_y	bullet off (miss)
Alien bullet touches the ship	box overlap with the ship	life_lost: lives - 1, or GAME OVER at 0
Alien bullet reaches the bottom	y + 14 >= 468	bullet off
Alien reaches the ship's row	y + 18 >= 440 for any alive alien	GAME OVER immediately
All aliens defeated	&defeated == 1	GAME WON

4. All together in one clock

All of the above lives in one clocked block. In one clock tick the code goes through the steps in the order shown below. The order matters because if two steps assign the same register in the same clock, the later one wins; that is why the “alien reached the ship's row” check is last.



Figure 6. Order of the steps inside the PLAY state.

Terms used

Term	Meaning
move_tick	One-clock pulse every 10 ms; all movement happens on it.
box	The rectangle occupied by an object; used for the touching test.
bullet_y	The bottom edge of the player's bullet (it is drawn 14 px upward from it).
cx, cy	The x and y of an alien's top-left corner in the hit test.
alive	Alien is on screen and not shot: present AND NOT defeated.
hit	Temporary flag: 1 when the player's bullet has just hit an alien.
defeated[k]	1 once alien k has been shot.
life_lost	1 for one clock when an alien bullet hits the ship.
touch	1 when a live alien has reached the ship's row.
top_y	The y where a missed player bullet gives up (top of the alien area).